

PROJECTREPORT

BANK SIMULATOR



PROJECT TITLE: CLI BANK SIMULATOR

COURSE TITLE: INTRODUCTION TO PROGRAM SOLVING
AND PROGRAMMING

COURSE CODE: CSA1021

NAME: YOGESH M

REGISTRATION NUMBER: 26BAI10575

COURSE TYPE: FLIPPED COURSE

1. PROJECT OVERVIEW

In our daily lives, we use banking apps constantly, but we rarely stop thinking about the logic that prevents a balance from disappearing or ensures a withdrawal doesn't exceed the available funds. This project, the **Bank Simulator**, is a practical exploration into the "engine" of a financial application.

Instead of building a fancy website, I focused on creating a lightweight, Command Line Interface (CLI) tool. It handles the core essentials: letting a user open an account, deposit money, and take money out, all while keeping a permanent record on the computer's hard drive.

2. THE PROBLEM & OBJECTIVE

Most beginner programming projects lose all their data as soon as you close the window. For a banking app, that is a failure. My main objectives were:

- **Data Persistence:** I wanted to make sure that if I deposited ₹ 500 today, it would still be there when I opened the program tomorrow.
- **Input Reliability:** In the real world, users make mistakes. I needed a system that wouldn't crash if someone typed "ten" instead of "10."
- **Minimalism:** By using only the Python Standard Library, I aimed to create a tool that is fast, secure, and doesn't require any internet connection or heavy installations.

3. MY METHODOLOGY & TECHNICAL CHOICES

The project follows a **modular, procedural programming approach** designed to simulate real-world banking operations through a Command Line Interface (CLI). The application is structured around single-responsibility functions that execute specific transactional tasks within an interactive runtime loop.

Key Technical Decisions:

- **In-Memory Hash Map for Fast Access:** Keying accounts by account number inside a dictionary allows instant verification (`account number in accounts`) before executing any debit or credit action.
- **Dynamic Log Appending:** Using `list.Append()` inside each account's history field allows the system to maintain an expandable audit trail without pre-defining log limits.
- **Defensive Validation Logic:** Strict conditional gates prevent invalid operations—such

as withdrawing more funds than the available balance or accessing missing accounts — ensuring system stability.

-

4. CORE SYSTEM FEATURES

The project is built around four main "User Stories":

The Console-Based Banking Simulator includes five key features designed to perform essential banking operations through a Command Line Interface (CLI):

1. Account Creation Module

- **Purpose:** Registers a new user account in the system storage.
- **Functionality:** Prompts for a unique Account Number, Account Holder Name, and an Initial Deposit amount.
- **Validation & Security:** Checks whether the entered account number already exists in the system to prevent duplicate records and data overwrites.

2. Deposit Processing Module

- **Purpose:** Credits money into an existing bank account.
- **Functionality:** Accepts the account number and deposit amount, automatically updates the account balance, and generates a deposit log.
- **Validation & Security:** Verifies account existence before attempting to add funds.

3. Withdrawal Processing Module

- **Purpose:** Debits money from an existing bank account.
- **Functionality:** Accepts the account number and requested withdrawal amount, deducts the amount from the current balance, and generates a withdrawal log.
- **Validation & Security:** Implements overdraft protection by checking if `amount > balance`. If requested funds exceed available funds, the transaction is rejected with an "Insufficient balance!" warning.

4. Real-Time Balance Inquiry Module

- **Purpose:** Retrieves current account details.
- **Functionality:** Fetches and displays the Account Holder Name and current active balance corresponding to the requested Account Number.
- **Validation & Security:** Ensures non-existent account queries return a clean "Account not found!" message without causing system runtime errors.

5. Transaction Audit Logging Module

- **Purpose:** Maintains a chronological history of all activities for an account.
- **Functionality:** Automatically appends account opening details, deposits, and withdrawals to a dedicated list inside the account record.
- **Output:** Displays all recorded activities sequentially as a bulleted transaction statement.

5. SYSTEM PERFORMANCE

- **Speed:** Because there are no heavy graphics, the math and file saving happen almost instantly (under 0.1 seconds).
- **Stability:** By Using blocks, I made the program "Human-Proof." It catches typing errors and gives a polite error message instead of showing scary red code.

Portability: The entire project is just one .py file. It can be moved to any laptop and will run perfectly without needing to install anything extra.

6. WHAT I LEARNED FROM THIS PROJECT

Building this was a huge learning curve. It taught me that the most important part of coding isn't the "math"—it's the **logic flow**.

- I learned how to manage files and data structures effectively.
- I realized that "cleaning" user input (like using `.strip()` `.title()`) is essential to make a program feel professional.
- Most importantly, I learned how to debug my own work by anticipating where a user might make a mistake.

7. CONCLUSION

The **CLI Banking System** might look simple on the outside, but the logic inside is robust and reliable. It successfully simulates a real-world bank ledger and provides a solid foundation for adding more advanced features like passwords, transaction history, or even automated interest calculations in the future.

REFERENCES:

1. VITDYATHI COURSE
2. GOOGLE (DOUBT CLARIFICATION AND DOCUMENTATION)
3. PERR AND MENTOR FEEDBACK FROM

- Kannan Shanmugam
- Bavesh P
- Sanjay Kumar S
- Leo TM
- Raghul SK

```

1  accounts = {}
2
3
4  def create_account():
5      acc_num = input("Enter Account Number: ")
6
7      if acc_num in accounts:
8          print("Account already exists!")
9          return
10
11     name = input("Enter Account Holder Name: ")
12     balance = float(input("Enter Initial Deposit: "))
13
14     accounts[acc_num] = {
15         "name": name,
16         "balance": balance,
17         "history": [f"Account opened with {balance}"],
18     }
19     print(f"Account created successfully for {name}!")
20
21
22  def deposit_money():
23     acc_num = input("Enter Account Number: ")
24
25     if acc_num not in accounts:
26         print("Account not found!")
27         return
28
29     amount = float(input("Enter amount to deposit: "))
30     accounts[acc_num]["balance"] += amount
31     accounts[acc_num]["history"].append(f"Deposited {amount}")
32
33     print(f"Deposited {amount}. New Balance: {accounts[acc_num]['balance']}")
34
35

```

```

36 def withdraw_money():
37     acc_num = input("Enter Account Number: ")
38
39     if acc_num not in accounts:
40         print("Account not found!")
41         return
42
43     amount = float(input("Enter amount to withdraw: "))
44
45     if amount > accounts[acc_num]["balance"]:
46         print("Insufficient balance!")
47     else:
48         accounts[acc_num]["balance"] -= amount
49         accounts[acc_num]["history"].append(f"Withdrew {amount}")
50         print(
51             f"Withdrew {amount}. Remaining Balance: {accounts[acc_num]['balance']}"
52         )
53
54
55 def check_balance():
56     acc_num = input("Enter Account Number: ")
57
58     if acc_num not in accounts:
59         print("Account not found!")
60         return
61
62     print(f"Name: {accounts[acc_num]['name']}")
63     print(f"Current Balance: {accounts[acc_num]['balance']}")
64
65
66 def view_history():
67     acc_num = input("Enter Account Number: ")
68
69     if acc_num not in accounts:
70         print("Account not found!")
71         return

```

```

36 def withdraw_money():
37     acc_num = input("Enter Account Number: ")
38
39     if acc_num not in accounts:
40         print("Account not found!")
41         return
42
43     amount = float(input("Enter amount to withdraw: "))
44
45     if amount > accounts[acc_num]["balance"]:
46         print("Insufficient balance!")
47     else:
48         accounts[acc_num]["balance"] -= amount
49         accounts[acc_num]["history"].append(f"Withdrew {amount}")
50         print(
51             f"Withdrew {amount}, Remaining Balance: {accounts[acc_num]['balance']}"
52         )
53
54
55 def check_balance():
56     acc_num = input("Enter Account Number: ")
57
58     if acc_num not in accounts:
59         print("Account not found!")
60         return
61
62     print(f"Name: {accounts[acc_num]['name']}")
63     print(f"Current Balance: {accounts[acc_num]['balance']}")
64
65
66 def view_history():
67     acc_num = input("Enter Account Number: ")
68
69     if acc_num not in accounts:
70         print("Account not found!")
71         return

```

```
=== SIMPLE BANK MENU ===
1. Create Account
2. Deposit Money
3. Withdraw Money
4. Check Balance
5. View History
6. Exit
Enter choice (1-6): 1
Enter Account Number: 123456789
Enter Account Holder Name: Yogesh
Enter Initial Deposit: 1000000
Account created successfully for Yogesh!
```

```
=== SIMPLE BANK MENU ===
1. Create Account
2. Deposit Money
3. Withdraw Money
4. Check Balance
5. View History
6. Exit
Enter choice (1-6): 2
Enter Account Number: 123456789
Enter amount to deposit: 10000
Deposited 10000.0. New Balance: 1010000.0
```

```
=== SIMPLE BANK MENU ===
1. Create Account
2. Deposit Money
3. Withdraw Money
4. Check Balance
5. View History
6. Exit
Enter choice (1-6): 4
Enter Account Number: 123456789
Name: Yogesh
Current Balance: 1010000.0
```

CORE SYSTEM FEATURES	
1. Account Creation	--> Generates new user account records
2. Deposit Processing	--> Credits funds and updates balance
3. Withdrawal Processing	--> Validates & debits available funds
4. Balance Inquiry	--> Fetches real-time account status
5. Audit Log History	--> Displays sequential transaction logs

